

Motivation Efficient VLA inference without retraining



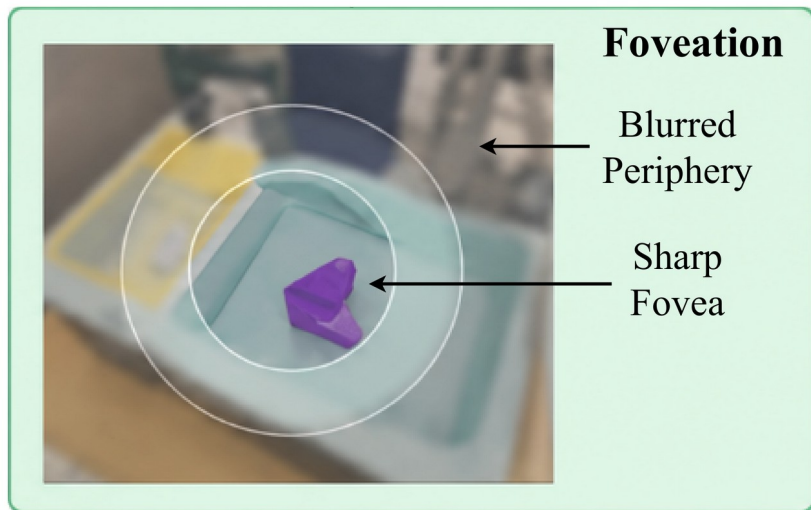
The control loop of a frozen VLA policy (Fig. 2 of the paper). Green, orange, purple, and blue boxes mark where the five tricks enter; Redundancy Control is action repeat.

[1] VLA inference is slow. Tens to hundreds of milliseconds per step (60 to 450 ms in our runs), which limits how often the policy can observe, decide, and correct.

[2] Gains are reported in isolation. Training-free tricks are typically evaluated for a specific trick, backbone, benchmark, and implementation, so transfer across models is unknown.

Do the gains transfer across VLA models and environments, **or are they configuration-dependent?**

Trick 1 Visual foveation



- Sharp central disc, progressively blurred periphery
- Keep ratio 20% or 50%; token count unchanged
- Tests whether high-frequency detail is needed only near the objects and the gripper

What to see

Original

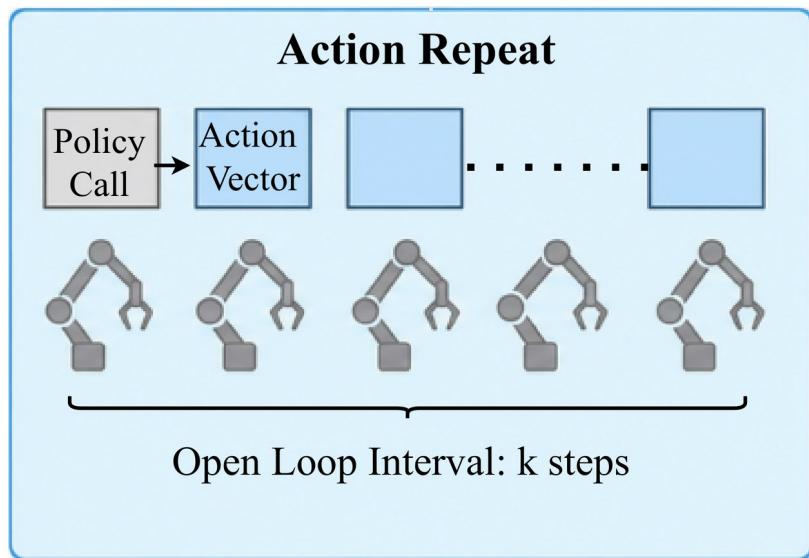


Foveation 50%



CogACT, WidowX, "Carrot on the plate", same episode, 2× speed.
Green = success, red = failure.

Trick 2 Action repeat



- Hold each predicted action for $k = 2$ or 4 steps, about one call per k steps
- Cuts policy calls by about k , but lengthens the open-loop interval
- Fixed reduction, no state check: a speed-oriented baseline

When to act

Original

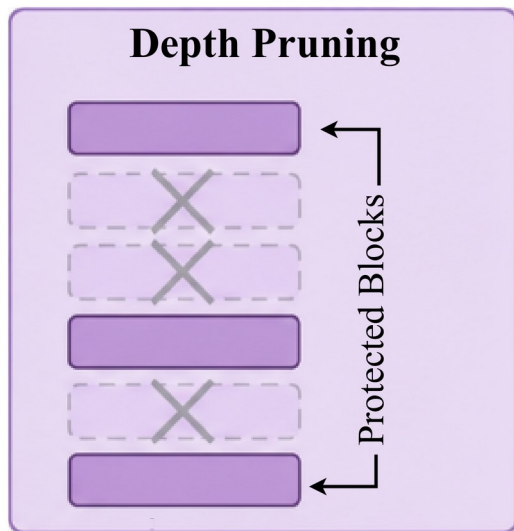


Action repeat 2



CogACT, WidowX, "Spoon on the towel", same episode, 2× speed.
Green = success, red = failure.

Trick 3 Depth pruning



- Score each decoder block by Block Influence
- Remove 1, 2, or 4 low-influence blocks; none adjacent, early blocks and the final block kept
- Fixed once from calibration data, not adapted at test time

How much to compute

Original



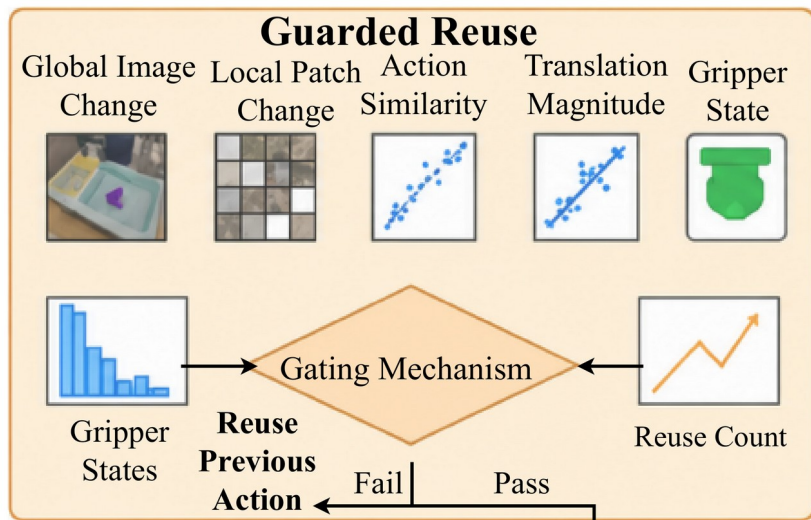
Depth pruning 1



CogACT, WidowX, "Carrot on the plate", same episode, 2× speed.
Green = success, red = failure.

Trick 4 Guarded action reuse

When to act



Original



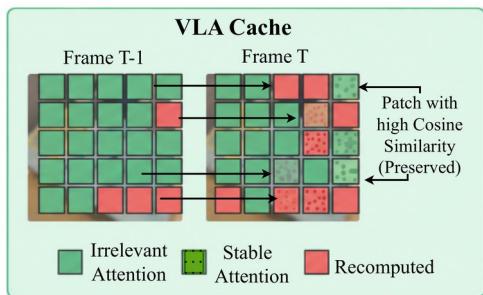
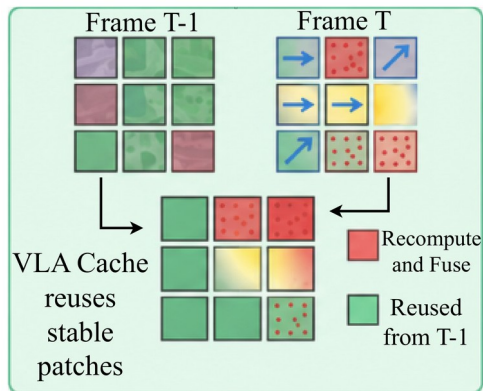
Reuse strict



CogACT, WidowX, "Eggplant in the basket", same episode, 2× speed.
Green = success, red = failure.

- Reuse the previous action only when all gates pass: small global and local image change, recent actions agree, translation not near zero, gripper unchanged, reuse count below a cap
- Presets: strict, moderate, aggressive

Trick 5 Temporal fusion



- Reuse visual tokens of stable patches from the previous call
- Recompute patches with motion, local structure, language attention, or in the interaction region
- Presets: motion-entropy, task-aware, conservative-adaptive

How much to compute

Original



Fusion task-aware



CogACT, WidowX, "Spoon on the towel", same episode, 2× speed.
Green = success, red = failure.

Top: fusion mask. Bottom: VLA-Cache criterion.

Results: matched evaluation protocol

Backbone	WidowX	Fractal	LIBERO
CogACT (7.6B)	●	●	
OpenVLA (7B)	●	●	●
SpatialVLA (4B)	●	●	
CronusVLA (0.5B)	●	●	
UniVLA (8.5B)	●		●
MiniVLA (1B)	●		
SmolVLA (0.45B)			●

SimplerEnv WidowX: 4 tasks × 50 episodes. Google Robot/Fractal: 5 tasks × 50. LIBERO: Long, Goal, Object, Spatial, each suite a separate pair ($3 \times 4 = 12$), giving 22 pairs in total.

22

backbone ×
environment pairs

13 + 1

trick settings +
original, per pair

286

trick settings (13×22), all paired with
the original

- Official checkpoints, no retraining
- Same task instances and initial states as the original
- Latency = wall-clock per environment step
- Exact two-sided McNemar test on paired episodes
- 4 × RTX 5090; tables report the best setting per trick

CogACT on SimplerEnv WidowX: "Carrot on the plate"

All 14 configurations on the same episode (same initial state), played at 2× speed. Green = success, red = failure (badge in each clip).

Original



Foveation 20%



Foveation 50%



Action repeat 2



Action repeat 4



Depth pruning 1



Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



Fusion motion/entropy



Fusion task-aware



Fusion conservative



CogACT on SimplerEnv WidowX: "Eggplant in the basket"

All 14 configurations on the same episode (same initial state), played at 3× speed. Green = success, red = failure (badge in each clip).

Original



Foveation 20%



Foveation 50%



Action repeat 2



Action repeat 4



Depth pruning 1



Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



Fusion motion/entropy



Fusion task-aware



Fusion conservative



CogACT on SimplerEnv WidowX: "Spoon on the towel"

All 14 configurations on the same episode (same initial state), played at 2× speed. Green = success, red = failure (badge in each clip).

Original



Foveation 20%



Foveation 50%



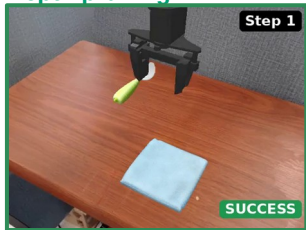
Action repeat 2



Action repeat 4



Depth pruning 1



Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



Fusion motion/entropy



Fusion task-aware



Fusion conservative



OpenVLA on Google Robot / Fractal: "Close the drawer"

All 12 configurations on the same episode (same initial state), played at 2× speed. Green = success, red = failure (badge in each clip).

Original



Foveation 20%



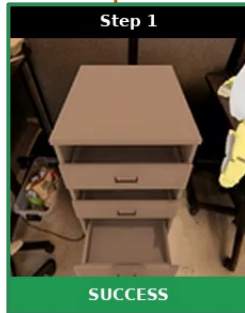
Foveation 50%



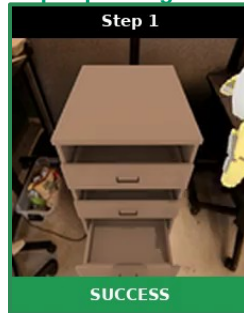
Action repeat 2



Action repeat 4



Depth pruning 1



Depth pruning 2



Depth pruning 4



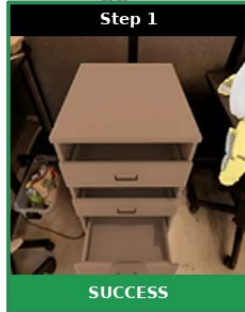
Reuse strict



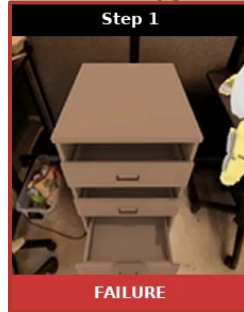
Reuse moderate



Reuse aggressive

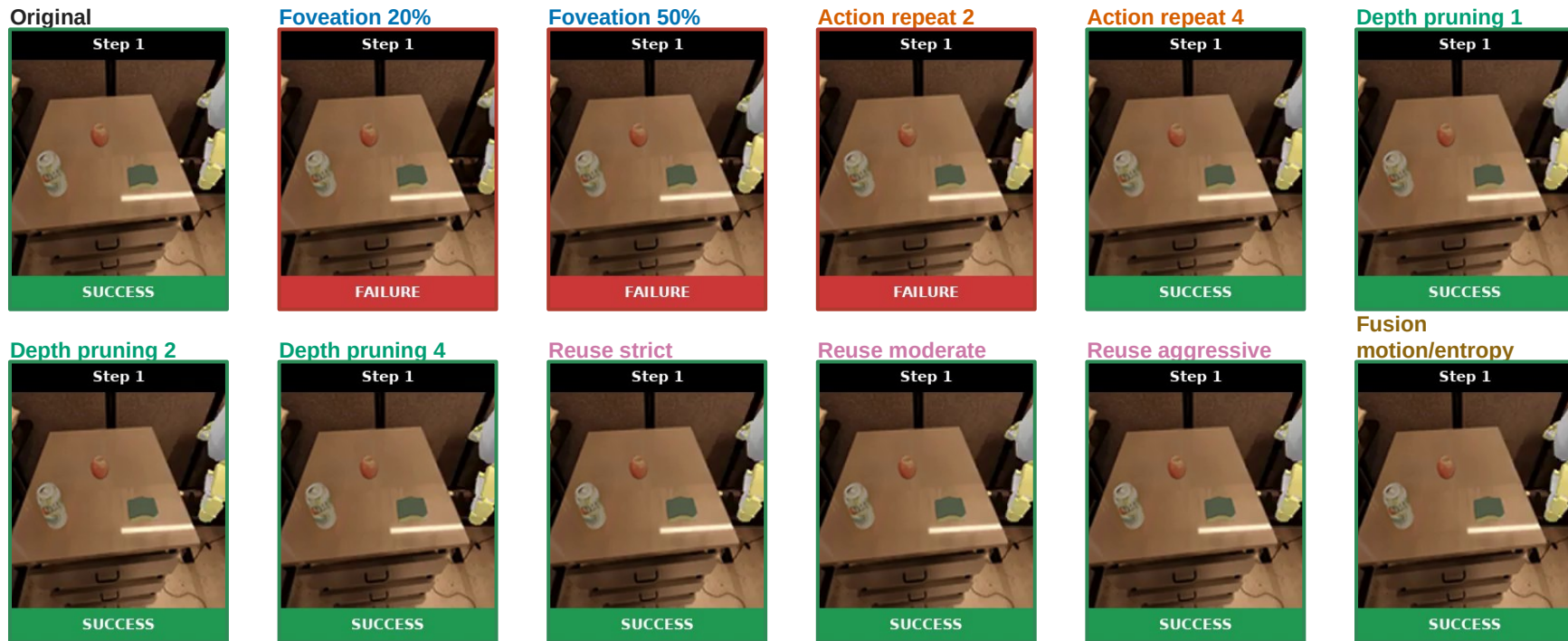


Fusion
motion/entropy



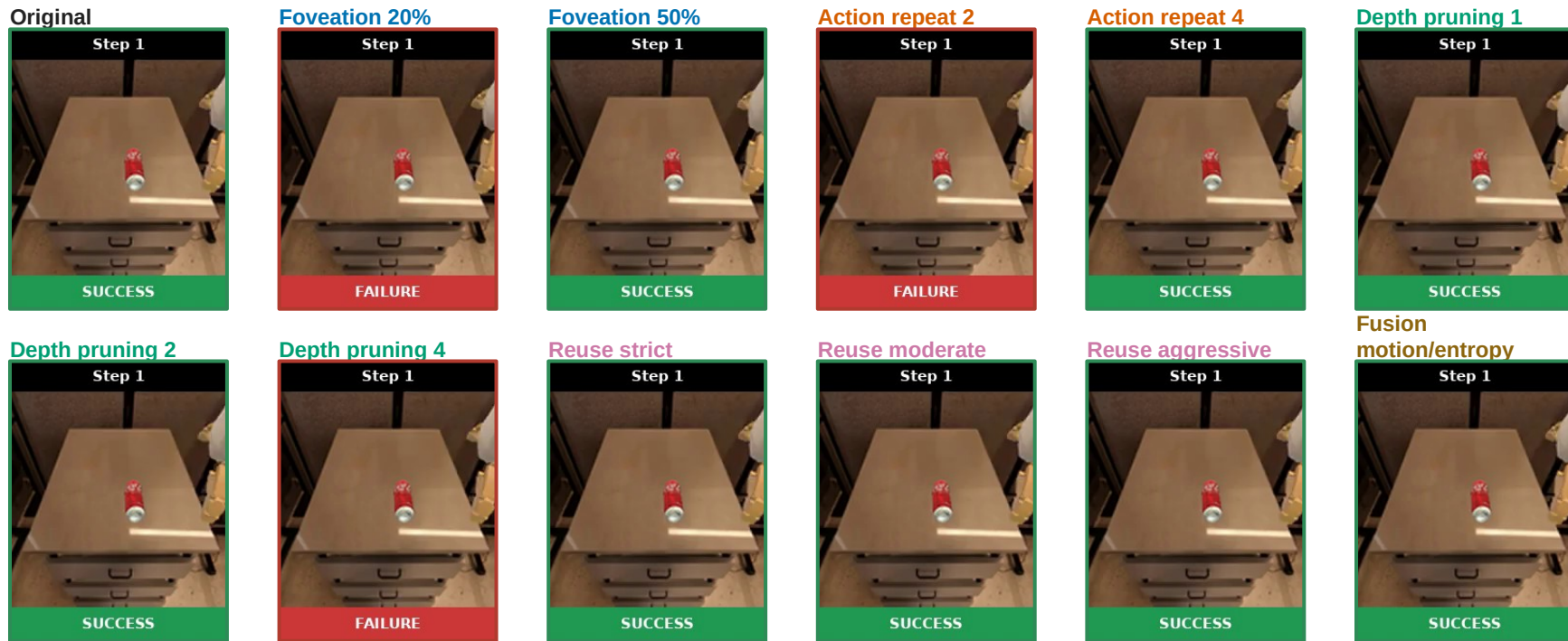
OpenVLA on Google Robot / Fractal: "Move the object near another"

All 12 configurations on the same episode (same initial state), played at 2× speed. Green = success, red = failure (badge in each clip).

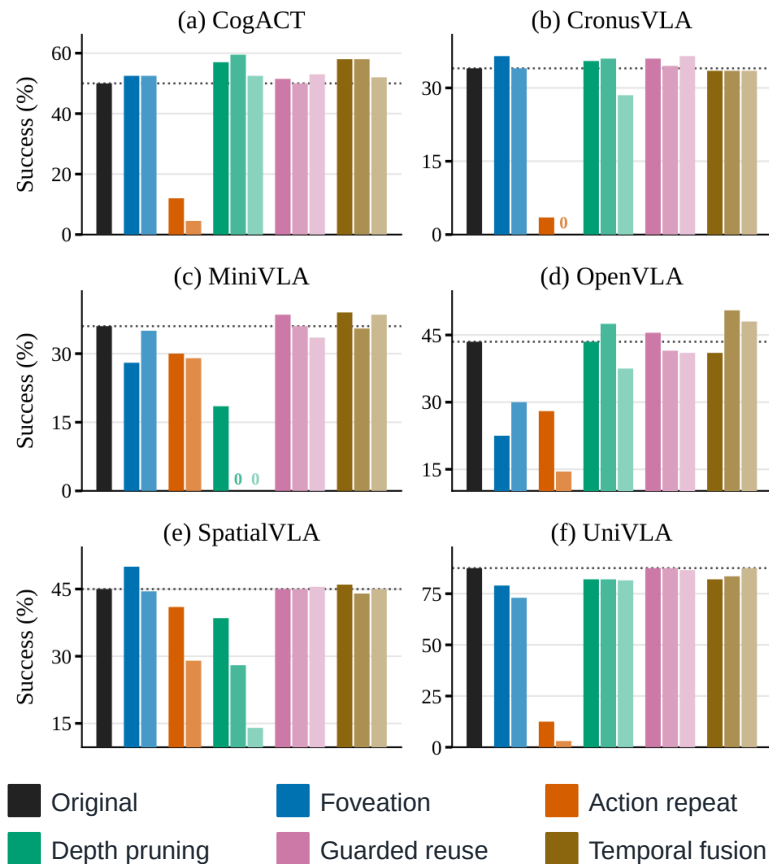


OpenVLA on Google Robot / Fractal: "Pick the coke can"

All 12 configurations on the same episode (same initial state), played at 2× speed. Green = success, red = failure (badge in each clip).



Which tricks preserve success?



Q1 Success

Guarded reuse and temporal fusion are the safest for preserving success.

Selected backbones also gain:

Foveation, WidowX: CogACT +2.5 · SpatialVLA +5.0 · CronusVLA +2.5

Depth pruning, WidowX: CogACT +9.5

Backbone-dependent:

Foveation, LIBERO Goal: UniVLA +7.0 vs SmoIVLA -13.0

Action repeat: mainly a speed baseline

Fig. 1: success per configuration, six WidowX backbones (14 bars each, darker to lighter within a trick). Dotted line = original policy, 0 = no successful episodes. Deltas from Tables I and II, best setting per trick.

Which tricks are actually faster?

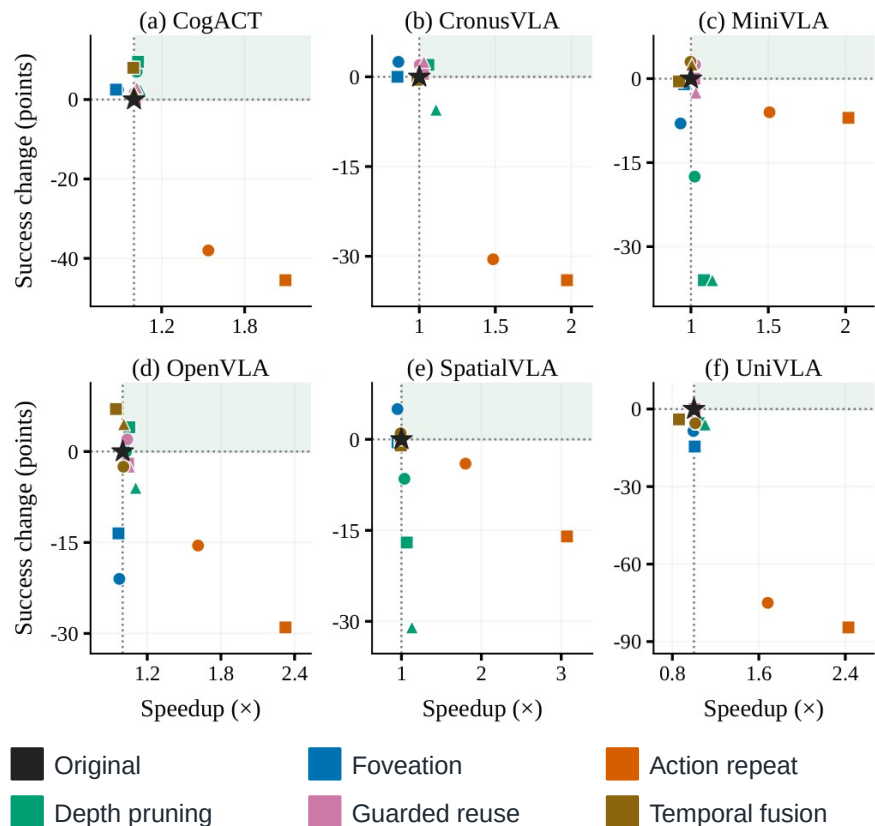


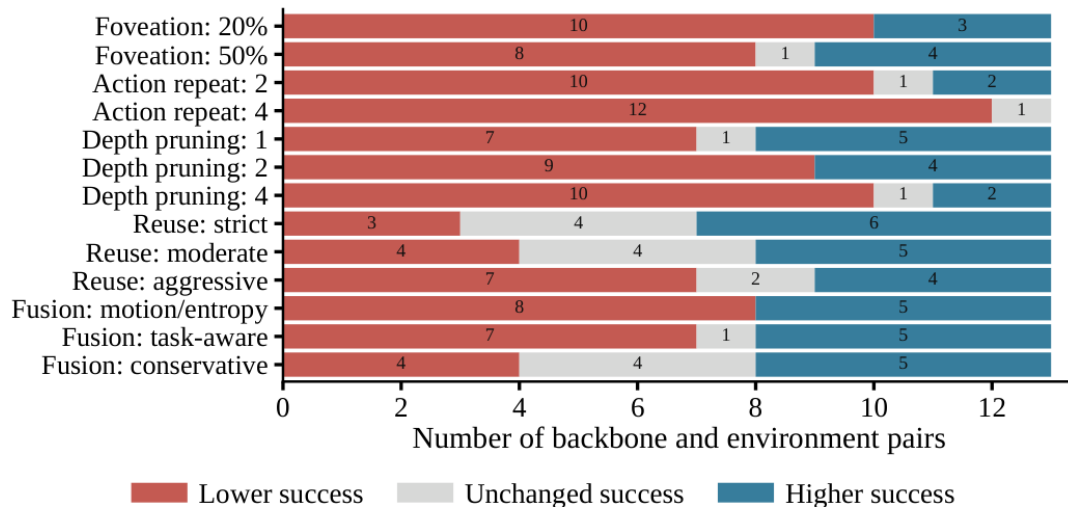
Fig. 4, SimplerEnv WidowX. Star = original, shaded = faster and no worse. Colour = trick; circle, square, triangle = first, second, third setting of that trick.

Q2 Speed

Action repeat gives the largest speedup, depth pruning gives decoder-level savings, guarded reuse gives the safest trade-off.

Latency, ms per step	Original → trick
Repeat · SpatialVLA WidowX	423.5 → 235.1
Repeat · OpenVLA LIBERO Long	171.6 → 93.2
Pruning · OpenVLA WidowX	210.2 → 199.4
Reuse · SmolVLA LIBERO Long	289.5 → 268.9

Does the effect transfer across models?



For each setting: how many of 13 backbone and environment pairs got lower, unchanged, or higher success. LIBERO episodes are pooled over the four suites, so 13 pairs stand for the 22.

Q3 Transfer

Effects are a joint property of the trick, the backbone, and the task distribution.

Depth pruning, WidowX: CogACT +9.5 · MiniVLA -17.5

Foveation, LIBERO Goal: UniVLA +7.0 · SmoVLA -13.0

Action repeat: often harmful on WidowX, milder on Fractal for some backbones

No trick is plug-and-play: each one needs matched evaluation under the target backbone and environment.

Takeaways

Guarded reuse

Most reliable. Preserves success, modest speedup when the gates open.

Action repeat

Largest speedup, but the risk of long open-loop execution.

Depth pruning

Decoder-level savings, benefit depends on the backbone.

Foveation · Temporal fusion

Test how much the visual representation can change while the policy stays stable.

VLA efficiency is policy- and environment-dependent. Evaluate every trick under the target backbone and environment before applying it.